

CSA 0378-DATA STRUCTURES

15. SPLAY TREE

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int key;
7     struct Node *left, *right;
8 };
9
10 struct Node* newNode(int key)
11 {
12     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
13     node->key = key;
14     node->left = node->right = NULL;
15     return node;
16 }
17
18 struct Node* rightRotate(struct Node* x)
19 {
20     struct Node* y = x->left;
21     x->left = y->right;
22     y->right = x;
23     return y;
24 }
25
26 struct Node* leftRotate(struct Node* x)
27 {
28     struct Node* y = x->right;
29     x->right = y->left;
30     y->left = x;
31     return y;
32 }
33
34 struct Node* splay(struct Node* root, int key)
35 {
36     if(root == NULL || root->key == key)
```

```
Output
Inorder Traversal:
30 40 50 100 200

Root after searching 50: 50

=== Code Execution Successful ===
```

```
main.c
39 if(key < root->key)
40 {
41     if(root->left == NULL)
42         return root;
43
44     if(key < root->left->key)
45     {
46         root->left->left = splay(root->left->left, key);
47         root = rightRotate(root);
48     }
49     else if(key > root->left->key)
50     {
51         root->left->right = splay(root->left->right, key);
52
53         if(root->left->right != NULL)
54             root->left = leftRotate(root->left);
55     }
56
57     return (root->left == NULL) ? root : rightRotate(root);
58 }
59
60 else
61 {
62     if(root->right == NULL)
63         return root;
64
65     if(key > root->right->key)
66     {
67         root->right->right = splay(root->right->right, key);
68         root = leftRotate(root);
69     }
70     else if(key < root->right->key)
71     {
72         root->right->left = splay(root->right->left, key);
73
74         if(root->right->left != NULL)
75             root->right = rightRotate(root->right);
76     }
77
78     return (root->right == NULL) ? root : leftRotate(root);
79 }
```

```
Output
Inorder Traversal:
30 40 50 100 200

Root after searching 50: 50

=== Code Execution Successful ===
```

```
main.c
81 struct Node* insert(struct Node* root, int key)
82 {
83     struct Node* node;
84
85     if(root == NULL)
86         return newNode(key);
87
88     root = splay(root, key);
89
90     if(root->key == key)
91         return root;
92
93     node = newNode(key);
94
95     if(key < root->key)
96     {
97         node->right = root;
98         node->left = root->left;
99         root->left = NULL;
100     }
101     else
102     {
103         node->left = root;
104         node->right = root->right;
105         root->right = NULL;
106     }
107
108     return node;
109 }
110
111 struct Node* search(struct Node* root, int key)
112 {
113     return splay(root, key);
114 }
115
116 void inorder(struct Node* root)
```

```
Output
Inorder Traversal:
30 40 50 100 200

Root after searching 50: 50

=== Code Execution Successful ===
```

```
main.c
109 }
110
111 struct Node* search(struct Node* root, int key)
112 {
113     return splay(root, key);
114 }
115
116 void inorder(struct Node* root)
117 {
118     if(root)
119     {
120         inorder(root->left);
121         printf("%d ", root->key);
122         inorder(root->right);
123     }
124 }
125
126 int main()
127 {
128     struct Node* root = NULL;
129
130     root = insert(root, 100);
131     root = insert(root, 50);
132     root = insert(root, 200);
133     root = insert(root, 40);
134     root = insert(root, 30);
135
136     printf("Inorder Traversal:\n");
137     inorder(root);
138
139     root = search(root, 50);
140
141     printf("\n\nRoot after searching 50: %d\n", root->key);
142
143     return 0;
144 }
```

```
Output
Inorder Traversal:
30 40 50 100 200

Root after searching 50: 50

=== Code Execution Successful ===
```